Output :

[1]:
```
# ### BASIC DATA TYPES
```

[2]:
```
print("Hello World")
```

```
Hello World
```

[3]:
```
a = 5
print(a)
print(type(a))
f = 1.5
print(f)
print(type(f))
s = "Hello"
print(s)
print(type(s))
```

```
5
<class 'int'>
1.5
<class 'float'>
Hello
<class 'str'>
```

[4]:
```
t = True
f = False
print(t, f)
print(type(t))
print(type(f))
```

```
True False
<class 'bool'>
<class 'bool'>
```

[5]:
```
t = 5 + 3j
print(t)
print(type(t))
```

```
(5+3j)
<class 'complex'>
```

[6]:
```
# ### ARITHMETIC OPERATORS
```

[7]:
```
a = 7
b = 3
sum = a + b
print("Sum=", sum)
diff = a - b
print("Difference=", diff)
pro = a * b
print("Product=", pro)
quo = a / b
print("Quotient=", quo)
iquo = a // b
print("Floor Division=", iquo)
rem = a % b
print("Remainder=", rem)
pow = a ** b
print("Power=", pow)
```

```
Sum= 10
Difference= 4
Product= 21
Quotient= 2.3333333333333335
Floor Division= 2
Remainder= 1
Power= 343
```

EXPERIMENT NO: 01

Aim: Program to implement python basics — Datatypes, Operators, Containers, Function

Description:

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991. It is used for web development (server-side), software development, mathematics, system scripting.

Data Types:

- **Integers (int):** whole numbers (eg: 1, 2, 3)
- **Floats (float):** decimal numbers (eg: 3.14, 0.5)
- **Strings (str):** sequence of characters (eg: "hello", 'hello')
- **Boolean (bool):** True or False values
- **List (list):** ordered collections of values (eg: [1, 2, 3], ['a', 'b', 'c'])
- **Tuple (tuple):** ordered, immutable collections of values (eg: (1, 2, 3), ('a', 'b', 'c'))
- **Dictionary (dict):** unordered collections of key-value pairs *(*eg: (1, 2, 3), (“a”, “b”, “c”))

Operators :

Operators are used to perform operations on variables and values. Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

[ ]:
```
# ### BOOLOEAN OPERATIONS
```

[1]:
```
t = True
f = False
print(t, f)
p = 5 > 3
print(p)
```

```
True False
True
```

[2]:
```
q = -1 < -12.5
print(q)
```

```
False
```

[3]:
```
print(p and q)
print(p or p)
print(not q)
```

```
False
True
True
```

[4]:
```
# ### STRING OPERATIONS
```

[5]:
```
s = 'Hello'
u = "Hello"
print(s)
print(u)
```

```
Hello
Hello
```

[6]:
```
s1 = "python"
s2 = 'world'
s3 = s1 + ' ' + s2
print(s3)
```

```
python world
```

[7]:
```
s3 = '%s %s %d' % (s1, s2, 1011)
print(s3)
```

```
python world 1011
```

[8]:
```
print(len(s3))
```

```
17
```

[9]:
```
print(s3.upper())
```

```
PYTHON WORLD 1011
```

[10]:
```
print(s3.capitalize())
```

```
Python world 1011
```

[12]:
```
print(s3.lower())
```

```
python world 1011
```

[13]:
```
print('hello world how are you'.split(' '))
```

```
['hello', 'world', 'how', 'are', 'you']
```

[14]:
```
print('book'.replace('o', 'e'))
```

```
beek
```

[15]:
```
word = 'jewellery'
print(word.find('well'))
print(word.find('is'))
```

```
2
-1
```

[ ]:
```
# ### CONTROL STRUCTURES
```

[17]:
```
num = 123
if num > 99 and num < 1000:
    print('3 DIGIT')
else:
    print('NOT 3 DIGIT')
```

```
3 DIGIT
```

[18]:
```
res = input('Are u familiar with python? ')
if res.upper() == 'YES':
    print("You can skip this course")
elif res.upper() == 'NO':
    print("You are at the right place")
else:
    print("Sorry, Wrong input!")
```

```
Are u familiar with python?  yes
You can skip this course
```

[ ]:
```
# ### CONTAINERS
```

[ ]:
```
#### LIST
```

[6]:
```
mylist = ['a', 'b', 1, 1.2, True]
print(mylist)
mylist.append("new")
print(mylist)
print(mylist.pop())
mylist.insert(2, 'new')
print(mylist)
mylist.remove('new')
print(mylist)
```

```
['a', 'b', 1, 1.2, True]
['a', 'b', 1, 1.2, True, 'new']
new
['a', 'b', 'new', 1, 1.2, True]
['a', 'b', 1, 1.2, True]
```

[7]:
```
b = [1, 2, 3]
print(b)
mylist.append(b)
print(mylist)
mylist.remove(b)
print(mylist)
mylist.extend(b)
print(mylist)
a = [2, 3, 1, 4, 5]
a.sort()
print(a)
print(list('hello'))
```

```
[1, 2, 3]
['a', 'b', 1, 1.2, True, [1, 2, 3]]
['a', 'b', 1, 1.2, True]
['a', 'b', 1, 1.2, True, 1, 2, 3]
[1, 2, 3, 4, 5]
['h', 'e', 'l', 'l', 'o']
```

[ ]:
```
#### DICTIONARY
```

[8]:
```
person = {'name': 'Manu', 'age': 28}
print(person['name'])
print('name' in person)
print('sex' in person)
person['sex'] = 'male'
print(person)
for item in person:
    print(item, person[item])
for (key, value) in person.items():
    print(key.capitalize(), '\t:\t', value)
print(person.keys())
```

```
Manu
True
False
{'name': 'Manu', 'age': 28, 'sex': 'male'}
name Manu
age 28
sex male
Name     :        Manu
Age      :        28
Sex      :        male
dict_keys(['name', 'age', 'sex'])
```

[9]:
```
my_set = {1, 2, 3, 4, 5}
for item in my_set:
    print(item)
```

```
1
2
3
4
5
```

Containers

Containers in Python are data structures that can store multiple values. Here are the main types of containers:

1. Lists

- Ordered collection of values
- Mutable (can be modified)
- Defined using square brackets []
- Elements can be accessed using indexing (eg, my_list[0])

2. Tuples

- Ordered, immutable collection of values
- Defined using parentheses ()
- Elements can be accessed using indexing (eg, my_tuple[0])

3. Dictionaries

- Unordered collection of key-value pairs
- Mutable (can be modified)
- Defined using curly brackets {}
- Elements can be accessed using key (eg, my_dict['key'])

4. Sets

- Unordered collection of unique values
- Mutable (can be modified)
- Defined using set() function or {} syntax
- Elements can be accessed using iteration (eg, for item in my_set:)

[ ]:
```
#### LIST SLICING
```

[10]:
```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(numbers[1], numbers[-1])
sliced = numbers[5:11]
print(sliced)
slice1 = numbers[5:]
print(slice1)
Sliced = numbers[:7]
print(Sliced)
slice2 = numbers[-2:]
print(slice2)
```

```
1 10
[5, 6, 7, 8, 9, 10]
[5, 6, 7, 8, 9, 10]
[0, 1, 2, 3, 4, 5, 6]
[9, 10]
```

[ ]:
```
#### LIST COMPREHENSION
```

[11]:
```
numbers = list(range(1, 8))
print(numbers)
# Equivalent standard loop setup
square = []
for i in numbers:
    square.append(pow(i, 2))
print(square)
# Equivalent list comprehension syntax
square = [x**2 for x in numbers]
print(square)
# List comprehension with condition (filtering odd numbers)
odd_square = [x**2 for x in numbers if x % 2 != 0]
print(odd_square)
# Cartesian product excluding self-pairs
A = [4, 6, 8, 9]
AxA = [(a, b) for a in A for b in A if a != b]
print(AxA)
```

```
[1, 2, 3, 4, 5, 6, 7]
[1, 4, 9, 16, 25, 36, 49]
[1, 4, 9, 16, 25, 36, 49]
[1, 9, 25, 49]
[(4, 6), (4, 8), (4, 9), (6, 4), (6, 8), (6, 9), (8, 4), (8, 6), (8, 9), (9, 4), (9, 6), (9, 8)]
```

[12]:
```
numbers = [1, 2, 3, 4, 5]
double_numbers = [x * 2 for x in numbers]
print(double_numbers)
```

```
[2, 4, 6, 8, 10]
```

[15]:
```
even_numbers = [x for x in numbers if x % 2 == 0]
print(even_numbers)
```

```
[2, 4]
```

[ ]:
```
### Nested List Comprehension
```

[13]:
```
matrix = [[1, 2], [3, 4], [5, 6]]
flattened = [x for sublist in matrix for x in sublist]
print(flattened)
```

```
[1, 2, 3, 4, 5, 6]
```

List Slicing:

List slicing is a powerful feature in programming languages that allows you to extract a subset of elements from a list.

Basic Syntax: list_name[start_index : end_index : step_size]

- **start_index**: The starting index of the slice (inclusive)
- **end_index**: The ending index of the slice (exclusive)
- **step_size**: The increment between indices (default is 1)

List Comprehension Notes

Definition: List comprehension is a concise way to create lists in programming languages.

Basic Syntax: [expression for variable in iterable]

- **expression**: The operation to perform on each element.
- **variable**: The temporary variable for each element.
- **iterable**: The list, tuple, or other iterable to process.

[ ]:
```
# #### FUNCTIONS
```

[16]:
```
# FINDING THE SQUARE OF A NUMBER
def square(number):
    return pow(number, 2)
s = square(5)
print("square(5)= ", s)
# CHECK WHETHER PRIME
def isPrime(number):
    for factor in range(2, (number // 2) + 1):
        if number % factor == 0:
            return False
    return True
# PRIME NUMBERS IN A RANGE
number = int(input("Enter the number "))
print(isPrime(number))
def printPrimes(llimit, ulimit):
    for num in range(llimit, ulimit + 1):
        if isPrime(num) == True:
            print(num, end=' ')
printPrimes(5, 50)
```

```
square(5)=  25
Enter the number  25
False
5 7 11 13 17 19 23 29 31 37 41 43 47
```

[17]:
```
# SWAP TWO NUMBERS
def swap(x, y):
    t = x
    x = y
    y = t
    return x, y

a = 5
b = 7
a, b = swap(a, b)
print(a, b)
```

```
7 5
```

Functions:

Functions are reusable blocks of code that perform a specific task.

Defining a function:

Python

```
def function_name(parameters):
    # code to be executed
```

- **function_name**: Name of the function.
- **parameters**: Variables passed to the function.
- **code to be executed**: Statements that run when the function is called.

Function Types:

1. **Built-in Functions**: Provided by Python (eg., len(), print()).
2. **User-defined Functions**: Created by developers.
3. **Anonymous Functions**: Defined without a name (eg., lambda functions).
4. **Recursive Functions**: Call themselves.

Calling a function:

- **General Syntax:** function_name(argument1, argument2, ..., argumentN)
- **function_name**: Name of the function to call.
- **argument1, argument2, ..., argumentN**: Values passed to the function.

Result:

Program executed successfully. Output obtained and verified.

[ ]:
```
#### CREATING ARRAY USING NUMPY
```

[1]:
```
import numpy as np
```

[ ]:
```
#### CREATE A 1D ARRAY
```

[4]:
```
array=np.array([1,2,3,4,5])
print(array)
```

```
[1 2 3 4 5]
```

[5]:
```
array=np.array([[1,2],[3,4]])
print(array)
```

```
[[1 2]
 [3 4]]
```

[6]:
```
#### CREATING ARRAY FILLED WITH ZEROS
```

[7]:
```
import numpy as np
```

[8]:
```
#### CREATE A 1D ARRAY OF ZEROS
```

[9]:
```
array=np.zeros(5)
print(array)
```

```
[0. 0. 0. 0. 0.]
```

[ ]:
```
#### CREATE A 2D ARRAY OF ZEROS
```

[10]:
```
array=np.zeros((3,4))
print(array)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

[11]:
```
import numpy as np
ones=np.ones((2,3))
 #2x3 array of ones
print(ones)
print("\n")
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

[12]:
```
range_arr=np.arange(0,10,2)
#ARRAY FROM 0 TO 10 WITH STEP2
print(range_arr)
```

```
[0 2 4 6 8]
```

EXPERIMENT NO: 02

AIM: Familiarisation of NumPy Library

DESCRIPTION: NumPy (Numerical Python) is a library for working with arrays and mathematical operations in Python. It provides support for large, multi-dimensional arrays and matrices, and is the foundation of most scientific computing in Python.

Key Features of NumPy

1. **Multi-dimensional arrays:** NumPy's core feature is its ability to create and manipulate n-dimensional arrays, which are similar to lists in Python but offer most efficient storage and operations.

2. **Vectorized operations:** NumPy allows you to perform operations on entire arrays at once, making it much faster than working with Python lists.

3. **Mathematical functions:** NumPy provides an extensive set of mathematical functions to perform element-wise operations, linear algebra, random number.

4. **Integration with other libraries:** NumPy is designed to work seamlessly with other popular Python libraries for scientific computing such as Pandas, SciPy and Matplotlib.

Benefits of Using NumPy

1. **Efficient computation:** NumPy's vectorized operations make it much faster than working with Python lists, especially for large datasets.
2. **Convenient data structure:** NumPy arrays provide a convenient data structure for representing and manipulating numerical data.
3. **Improved code readability:** By using NumPy's high-level functions and operators, you can write more concise and readable code.
4. **Scalability:** NumPy can handle large datasets and perform complex computations efficiently, making it suitable for a wide range of applications.

Common Use Cases for NumPy

1. **Scientific computing:** NumPy is widely used in scientific computing for tasks such as numerical simulations, data analysis and visualization.

2. **Data analysis:** NumPy is used in data analysis to perform statistical computations, data cleaning and data transformation.

3. **Machine Learning:** NumPy is used in machine learning to implement algorithms and perform computations on large datasets.

4. **Image and signal processing:** NumPy is used in image and signal processing to perform tasks such as filtering, convolution and feature extraction.

[ ]:
```
#### BASIC OPERATIONS
```

[ ]:
```
#### INDEXING AND SLICING
```

[13]:
```
arr=np.array([1,2,3,4,5])
print(arr[0])
#ACCESSING THE FIRST ELEMENT
print(arr[1:3])
#SLICING THE ARRAY
```

```
1
[2 3]
```

[ ]:
```
#### BASIC MATHEMATICAL OPERATIONS
```

[14]:
```
arr1=np.array([1,2,3])
arr2=np.array([4,5,6])
#ADDITION
sum_arr=arr1+arr2
print(sum_arr)
 #SUBTRACTION
diff_arr=arr1-arr2
print(diff_arr)
#MULTIPLICATION (ELEMENT-WISE)
prod_arr=arr1*arr2
print(prod_arr)
#DIVISION (ELEMENT-WISE)
div_arr=arr1/arr2
print(div_arr)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

[ ]:
```
#### CHECK WHETHER THE ARRAYS ARE EQUAL
```

[15]:
```
import numpy as np
arr1=np.array([1,2,3,4])
arr2=np.array([1,2,3,4])
print(np.array_equal(arr1,arr2))
```

```
True
```

[17]:
```
####AGGREGATE FUNCTIONS
```

[18]:
```
arr=np.array([1,2,3,4,5])
print(np.sum(arr))
print(np.mean(arr))
print(np.max(arr))
print(np.min(arr))
```

```
15
3.0
5
1
```

[19]:
```
#### RESHAPING AND TRANSPOSING
```

[20]:
```
arr=np.array([[1,2],[3,4]])
reshaped_arr=arr.reshape(1,4)
print(reshaped_arr)
transposed_arr=np.transpose(arr)
print(transposed_arr)
```

```
[[1 2 3 4]]
[[1 3]
 [2 4]]
```

Result:

Program executed successfully. Output obtained and verified.

[ ]:
```
#### CREATING ARRAY USING NUMPY
```

[4]:
```
import numpy as np
# create a 2x2 matrix
matrix1 = np.array([[1, 3], [5, 7]])
print("2x2 Matrix:\n", matrix1)
# shape of matrix
print(np.shape(matrix1))
# size of the matrix
print(np.size(matrix1))
```

```
2x2 Matrix:
 [[1 3]
 [5 7]]
(2, 2)
4
```

[5]:
```
# create a 3x3 matrix
matrix2 = np.array([[2, 3, 5], [7, 14, 21], [1, 3, 5]])
print("\n3x3 Matrix:\n", matrix2)
# shape of matrix
print(np.shape(matrix2))
# size of the matrix
print(np.size(matrix2))
```

```
3x3 Matrix:
 [[ 2  3  5]
 [ 7 14 21]
 [ 1  3  5]]
(3, 3)
9
```

[ ]:
```
### MATRIX ADDITION AND SUBTRACTION
```

[6]:
```
# Matrix addition
matrix1 = np.array([[1, 12], [9, 15]])
matrix2 = np.array([[3, 6], [4, 8]])
result = matrix1 + matrix2
print("Sum =\n", result)
# Matrix subtraction
result = matrix1 - matrix2
print("Difference=\n", result)
```

```
Sum =
 [[ 4 18]
 [13 23]]
Difference=
 [[-2  6]
 [ 5  7]]
```

EXPERIMENT NO: 03

AIM: To implement Matrix operations using NumPy Library.

DESCRIPTION: Vectorization allows you to perform operations on entire matrices without explicit loops. NumPy provides efficient functions for various matrix operations. They are:

Matrix Creation:

1. numpy.array() — Create a matrix from a list
2. numpy.zeros() — Create a matrix filled with zeros
3. numpy.ones() — Create a matrix filled with ones
4. numpy.eye() — Create an identity matrix
5. numpy.random.rand() — Create a random matrix

Matrix Properties:

1. numpy.shape() — Get matrix shape
2. numpy.size() — Get matrix size
3. numpy.ndim() — Get matrix dimensions
4. numpy.dtype() — Get matrix data type

Matrix operations

1. numpy.add() — Element-wise addition
2. numpy.subtract() — Element-wise subtraction
3. numpy.multiply() — Element-wise multiplication
4. numpy.divide() — Element-wise division
5. numpy.dot() — Matrix multiplication
6. numpy.matmul() — Matrix multiplication (Python 3.5+)
7. numpy.transpose() — Matrix transpose

Matrix Properties:

1. numpy.shape() — Get matrix shape
2. numpy.size() — Get matrix size
3. numpy.ndim() — Get matrix dimensions
4. numpy.dtype() — Get matrix data type

[ ]:
```
### PERFORM MATRIX MULTIPLICATION
```

[7]:
```
# create two matrices
matrix1 = np.array([[1, 3], [5, 7]])
matrix2 = np.array([[2, 6], [4, 8]])
# calculate the dot product of the two matrices
result = np.dot(matrix1, matrix2)
print("matrix1 x matrix2: \n", result)
```

```
matrix1 x matrix2:
 [[14 30]
 [38 86]]
```

[ ]:
```
### TRANSPOSE OF A MATRIX
```

[8]:
```
# create a matrix
matrix1 = np.array([[1, 3], [5, 7]])
# get transpose of matrix1
result = np.transpose(matrix1)
print(result)
```

```
[[1 5]
 [3 7]]
```

[ ]:
```
### INVERSE OF A SQUARE MATRIX
```

[9]:
```
# create a 3x3 square matrix
matrix1 = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 5]])
# find inverse of matrix1
result = np.linalg.inv(matrix1)
print(result)
```

```
[[-1.83333333 -0.83333333  2.16666667]
 [ 1.5         0.83333333 -1.83333333]
 [-0.33333333 -0.33333333  0.66666667]]
```

[ ]:
```
### DETERMINANT OF A MATRIX
```

[10]:
```
# create a matrix
matrix1 = np.array([[1, 2, 3], [4, 5, 1], [2, 3, 4]])
# find determinant of matrix1
result = np.linalg.det(matrix1)
print(result)
```

```
-4.999999999999999
```

[ ]:
```
### SINGULAR VALUE DECOMPOSITION(SVD)
```

[11]:
```
matrix = np.array([[1, 2], [3, 4]])
U, s, Vh = np.linalg.svd(matrix)
print(U)
print(s)
print(Vh)
```

```
[[-0.40455358 -0.9145143 ]
 [-0.9145143   0.40455358]]
[5.4649857  0.36596619]
[[-0.57604844 -0.81741556]
 [ 0.81741556 -0.57604844]]
```

Matrix operations

1. numpy.add() — Element-wise addition
2. numpy.subtract() — Element-wise subtraction
3. numpy.multiply() — Element-wise multiplication
4. numpy.divide() — Element-wise division
5. numpy.dot() — Matrix multiplication
6. numpy.matmul() — Matrix multiplication (Python 3.5+)
7. numpy.transpose() — Matrix transpose

Linear algebra:

1. numpy.linalg.inv() : Matrix inverse
2. numpy.linalg.det() : Matrix determinant
3. numpy.linalg.solve() : Solve linear systems
4. numpy.linalg.eig() : Compute eigenvalues and eigenvectors
5. numpy.linalg.svd() : Singular value decomposition

Statistical Functions:

1. numpy.mean() : Compute mean
2. numpy.median() : Compute median
3. numpy.std() : Compute standard deviation
4. numpy.var() : Compute variance

Benefits of Vectorization

1. Faster computation
2. Reduced memory usage
3. Improved readability
4. Simplified code

Common Libraries

1. NumPy (Python)
2. SciPy (Python)
3. MATLAB
4. R

Singular Value Decomposition (SVD) in NumPy SVD is a factorization technique that decomposes a matrix into three matrices. SVD decomposition: $A = U \cdot \Sigma \cdot V^T$

where,

- A : Original matrix
- U : Left singular vectors (orthonormal)
- Σ : Singular values (diagonal matrix)
- V^T : Right singular vectors (orthonormal)

NumPy SVD Function: numpy.linalg.svd(A) returns:

- U : Left singular vectors
- s : Singular values
- Vh : Right singular vectors (conjugate transpose)

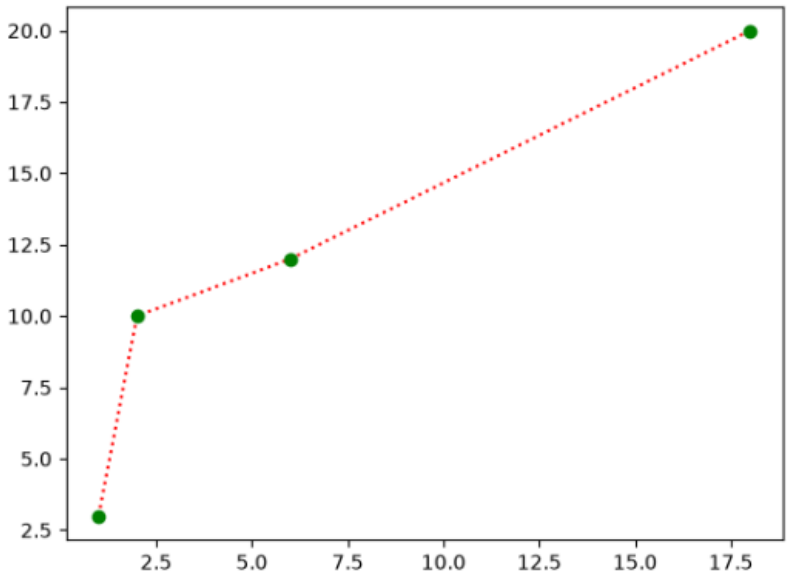
Result:

Program executed successfully. Output obtained and verified.

[ ]:
```
#Aim : Femiliarising matplotliband seabron libraries used for data visualization
```

[ ]:
```
# a) Draw a line in a diagram from position (1, 3) to (2, 10) then to (6, 12) and finally to position (18, 20).
(Mark each point with a beautiful green colour and set line colour to red and line style dotted).
```

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
# Define data points
xpoints = np.array([1, 2, 6, 18])
ypoints = np.array([3, 10, 12, 20])
# Create the plot
plt.plot(
    xpoints,
    ypoints,
    marker='o',         # Circle marker
    color='red',        # Line color
    mec='green',        # Marker edge color
    mfc='green',        # Marker face color
    linestyle='dotted'  # Dotted line
)
# Display the plot
plt.show()
```



EXPERIMENT NO: 04

AIM: Programs using matplotlib/seaborn for data visualisation

DESCRIPTION: Matplotlib is a popular Python library for creating static, animated and interactive visualizations.

Main Modules:

1. matplotlib.pyplot : Plotting functions
2. matplotlib.axes : Axis functions
3. matplotlib.figure : Figure functions
4. matplotlib.animation : Animation functions
5. matplotlib.widgets : Interactive widgets

Plotting functions:

1. plot() : Create a 2D line plot
2. scatter() : Create a scatter plot
3. bar() : Create a bar chart
4. hist() : Create a histogram
5. pie() : Create a pie chart

Chart Customization

1. title() : Set plot title
2. xlabel(), ylabel() : Set axis labels
3. legend() : Add legend
4. grid() : Display grid
5. axis() : Set axis limits

Axes Functions

1. subplot() : Create subplots
2. axes() : Create axes
3. axis('equal') : Set equal aspect ratio
4. axis('off') : Hide axis

[ ]:
```
# b) Consider the  following data
#   Programming Languages: Java, Python, PHP, JavaScript, C#, C++
#   Popularity: 22.2, 17.6, 8.8, 8, 7.7, 6.7
#   Write a Python program to display a bar chart of the popularity of programming languages.
```

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
data = {
    'Java': 22.2,
    'Python': 17.6,
    'PHP': 8.8,
    'JavaScript': 8,
    'C#': 7.7,
    'C++': 6.7
}
courses = list(data.keys())
values = list(data.values())
# Create a figure with custom size
fig = plt.figure(figsize=(10, 5))
# Creating the bar plot
plt.bar(courses, values, color='maroon', width=0.4)
plt.xlabel("Programming languages")
plt.ylabel("Popularity")
plt.title("Popularity of Programming Languages")
plt.show()
```

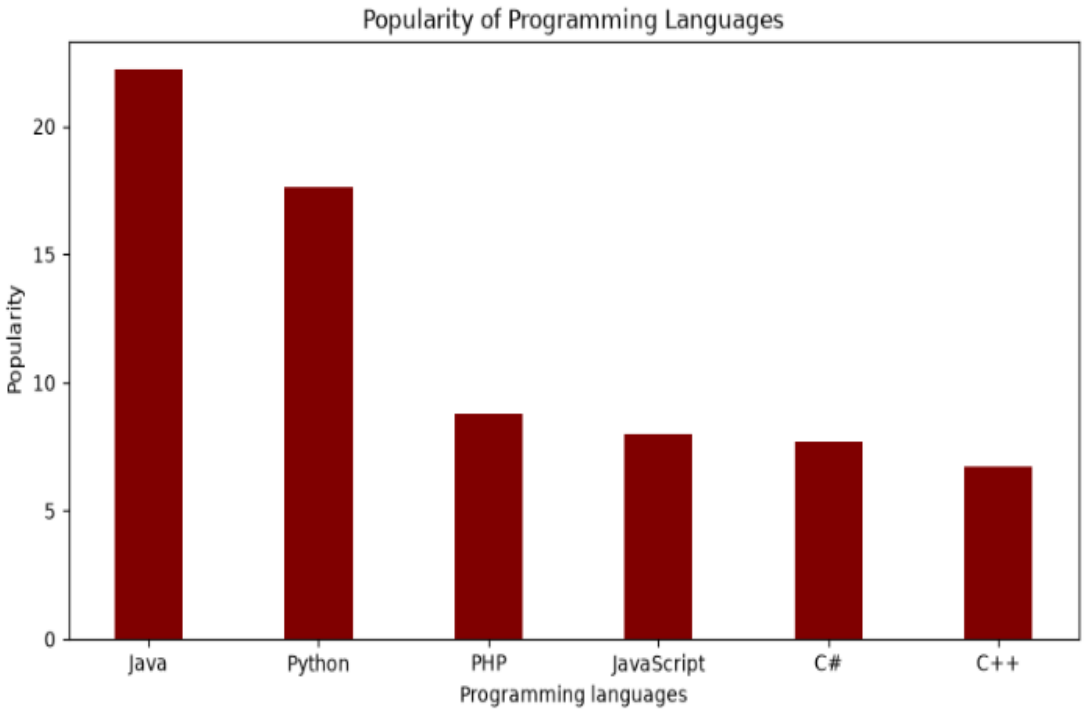


Figure Functions

1. figure() : Create a new figure
2. figsize() : Set figure size
3. dpi() : Set figure resolution
4. savefig() : Save figure to file

3D Plotting

1. plot3D() : Create a 3D line plot
2. scatter3D() : Create a 3D scatter plot
3. bar3d() : Create a 3D bar chart

Seaborn library :

Seaborn is a Python library built on top of Matplotlib for creating informative and attractive statistical graphics.

Statistical Functions:

1. regplot() : Regression plots
2. lmplot() : Linear regression plots
3. residplot() : Residual plots

Customization Functions:

1. set_style() : Set plot style
2. set_context() : Set plot context
3. palette() : Set color palette
4. color_palette() : Get color palette

Axes and Figure Functions:

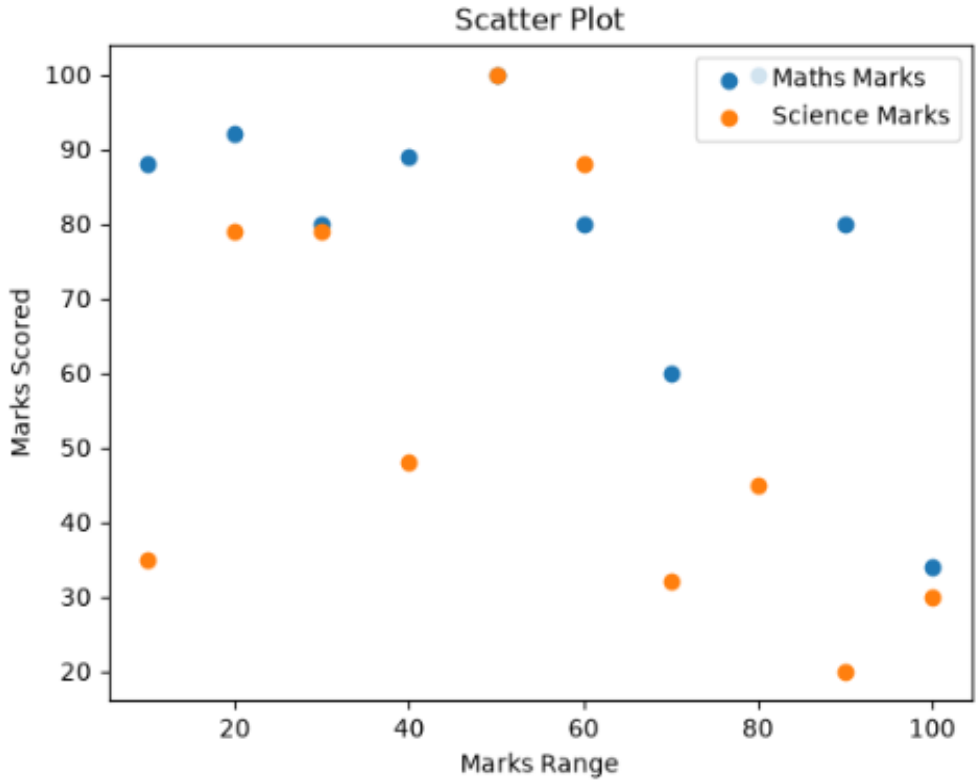
1. axes_style() : Set axes style
2. FacetGrid() : Create facet grid
3. PairGrid() : Create pair grid
4. JointGrid() : Create joint grid

Utility Functions:

1. load_dataset() : Load example dataset
2. get_dataset() : Get dataset information

[ ]:
```
### Write a Python program to draw a scatter plot comparing two subject marks
#   of Mathematics and Science. Use marks range of 10 to 100 for 10 students.
#   marks_range = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
#   math_marks = [88, 92, 80, 89, 100, 80, 60, 100, 80, 34]
#    science_marks = [35, 79, 79, 48, 100, 88, 32, 45, 20, 50]
```

[1]:
```
import matplotlib.pyplot as plt
# X-axis data
x = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
# Y-axis data for maths and science
m = [88, 92, 80, 89, 100, 80, 60, 100, 80, 34]
s = [35, 79, 79, 48, 100, 88, 32, 45, 20, 30]
# Create scatter plots
plt.scatter(x, m, label="Maths Marks")
plt.scatter(x, s, label="Science Marks")
# Add legend
plt.legend(loc='upper right')
# Add labels and title
plt.xlabel("Marks Range")
plt.ylabel("Marks Scored")
plt.title("Scatter Plot")
plt.show()
```



Plotting Functions:

barplot(), boxplot(), violinplot(), scatterplot(), lineplot(), pairplot(), jointplot(), distplot(), countplot()

Seaborn Modules :

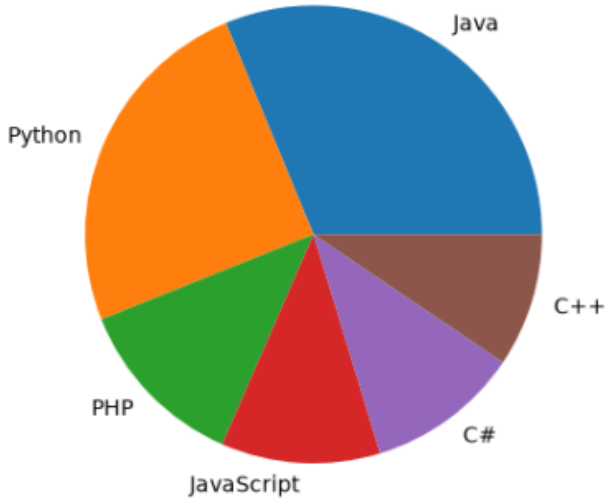
1. seaborn.plotting : Plotting functions
2. seaborn.axisgrid : Axis functions
3. seaborn.utils : Utility functions
4. seaborn.palettes : Color palettes

Benefits of using seaborn and matplotlib library

1. Data visualization and exploration
2. Publication-quality plots
3. Customizable plots
4. Integration with popular libraries (NumPy, Pandas)
5. Cross-platform compatibility
6. Large communities and extensive documentation
7. Free and open-source

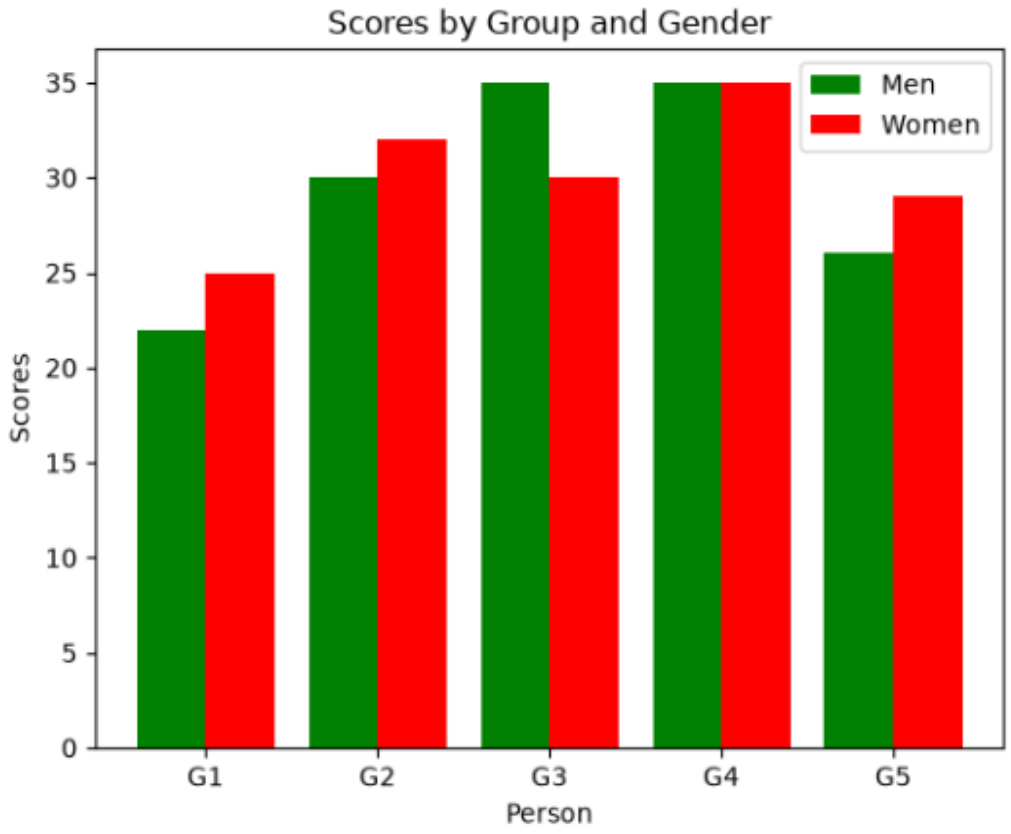
[ ]:
```
#   Write a Python programming to create a pie chart of the popularity of programming languages.
#   programming languages : "Java", "Python", "PHP", "JavaScript", "C#", "C++"
#   popularity            : 22.2,     17.6,    8.8,    8,           7.7,    6.7
```

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
# Data
y = np.array([22.2, 17.6, 8.8, 8, 7.7, 6.7])
mylabels = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
# Create pie chart
plt.pie(y, labels=mylabels)
plt.show()
```



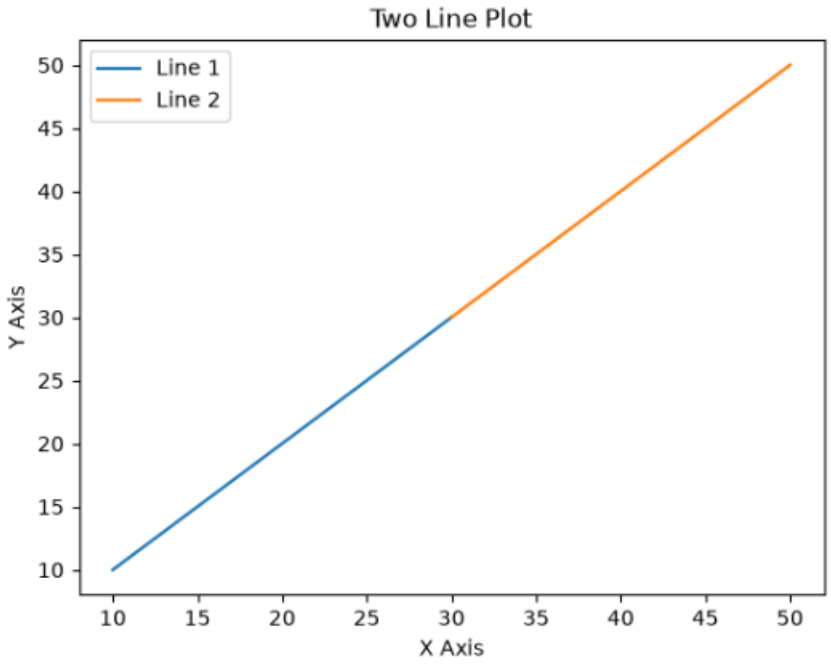
[ ]:
```
#   Write a Python program to create bar plot scores by group and gender.
#   Use multiple values on the same chart for men and women.
```

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
y1 = [22, 30, 35, 35, 26]
y2 = [25, 32, 30, 35, 29]
x_labels = ['G1', 'G2', 'G3', 'G4', 'G5']
x1 = np.arange(len(x_labels))
width = 0.40
# Create grouped bar chart
plt.bar(x1 - 0.2, y1, color='green', width=width, label='Men')
plt.bar(x1 + 0.2, y2, color='red', width=width, label='Women')
# Customize axes and labels
plt.xticks(x1, x_labels)
plt.xlabel("Person")
plt.ylabel("Scores")
plt.title("Scores by Group and Gender")
plt.legend()
plt.show()
```



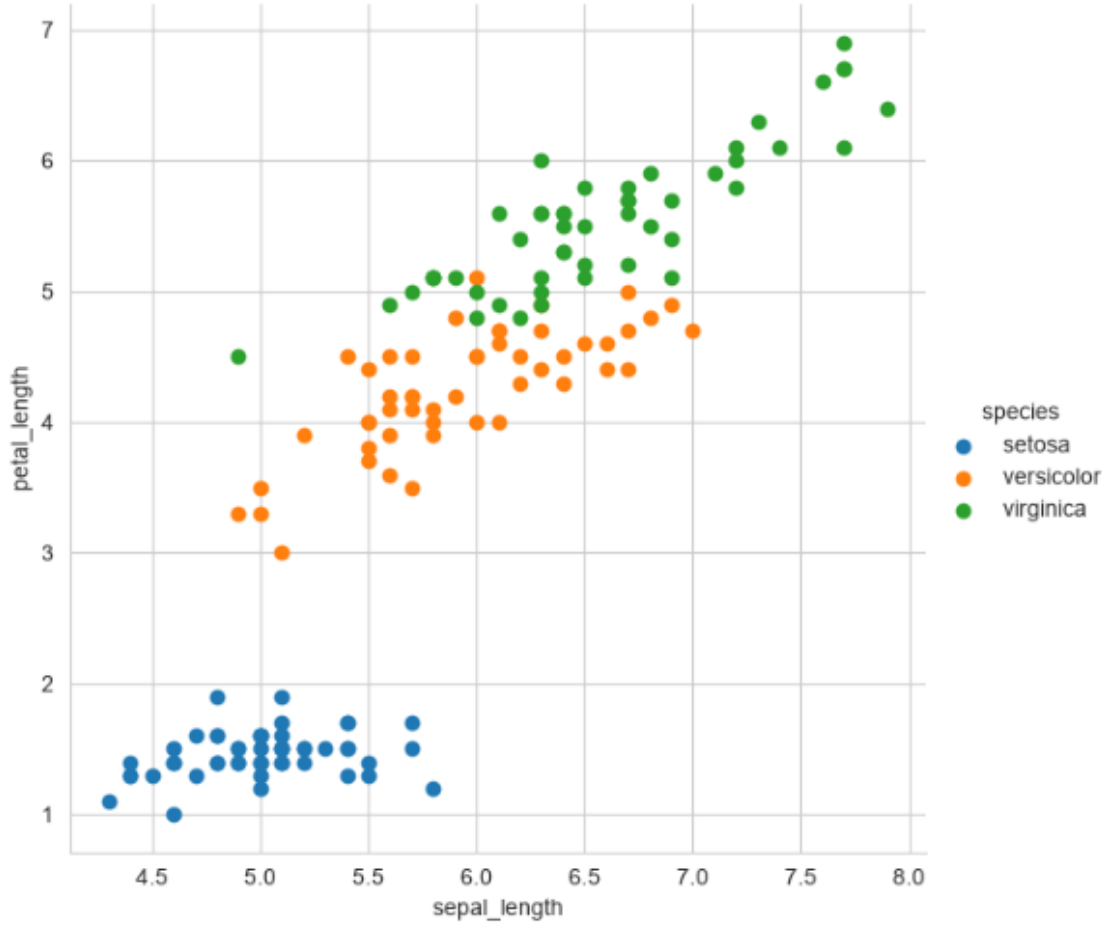
[ ]:
```
#  Write a Python program to plot two or more lines on same plot with suitable legends of each line.
```

[3]:
```
import matplotlib.pyplot as plt
# Line 1 data
x1 = [10, 20, 30]
y1 = [10, 20, 30]
plt.plot(x1, y1, label="Line 1")
# Line 2 data
x2 = [30, 40, 50]
y2 = [30, 40, 50]
plt.plot(x2, y2, label="Line 2")
# Labels and title
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('Two Line Plot')
plt.legend()
plt.show()
```



[ ]:
```
#  Create a scatter plot using Seaborn featuring functionality to visualize the Iris dataset.
```

[9]:
```
import matplotlib.pyplot as plt
import seaborn as sns
# Load dataset
iris = sns.load_dataset("iris")
# Set Seaborn style
sns.set_style("whitegrid")
# Create FacetGrid and plot
sns.FacetGrid(iris, hue="species", height=6) \
    .map(plt.scatter, "sepal_length", "petal_length") \
    .add_legend()
plt.show()
```



Result:

Program executed successfully. Output obtained and verified.

[3]:
```
import pandas as pd
from sklearn.datasets import load_iris
# Load the Iris dataset from scikit-learn
iris = load_iris()
# Create the DataFrame
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
# Add the target (species) column
iris_df['species'] = pd.Categorical.from_codes(
    iris.target, iris.target_names
)
# Now you can run your code
print(iris_df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.0 KB
None
```

[ ]:
```
#   Check for total number of missing values
```

[4]:
```
print(iris_df.isna().sum())
```

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
dtype: int64
```

[ ]:
```
### Group by species and calculate mean measurements
```

[5]:
```
print(iris_df.groupby('species').mean())
```

```
            sepal length (cm)  sepal width (cm)  petal length (cm)  \
species
setosa                  5.006             3.428              1.462
versicolor              5.936             2.770              4.260
virginica               6.588             2.974              5.552

            petal width (cm)
species
setosa                 0.246
versicolor             1.326
virginica              2.026
```

EXPERIMENT NO: 05

AIM: Programs to handle data using pandas.

DESCRIPTION: Pandas is a Python library used for working with datasets. It has functions for analyzing, cleaning, exploring, and manipulating data. Pandas allows us to analyze big data and make conclusions based on statistical theories. Pandas can clean messy data sets, and make them readable and relevant.

Here are some commonly used Pandas functions:

Data Manipulation

1. read_csv() : Read a CSV file.
2. to_csv() : Write a DataFrame to a CSV file.
3. head() : Display the first few rows of a DataFrame.
4. tail() : Display the last few rows of a DataFrame.
5. info() : Display information about a DataFrame.
6. describe() : Generate descriptive statistics for a DataFrame.
7. merge() : Merge two DataFrames.
8. join() : Join two DataFrames.
9. concat() : Concatenate two or more DataFrames.
10. groupby() : Group a DataFrame by one or more columns.

Data Cleaning

1. drop() : Remove rows or columns from a DataFrame.
2. dropna() : Remove rows with missing values.
3. fillna() : Fill missing values.
4. replace() : Replace values in a DataFrame.
5. unique() : Get unique values in a column.

[ ]:
```
# Get information about the DataFrame (data types, non-null counts)
```

[6]:
```
print(iris_df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype   
---  ------             --------------  -----   
 0   sepal length (cm)  150 non-null    float64 
 1   sepal width (cm)   150 non-null    float64 
 2   petal length (cm)  150 non-null    float64 
 3   petal width (cm)   150 non-null    float64 
 4   species            150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.0 KB
None
```

[ ]:
```
# Check for missing values
```

[7]:
```
print(iris_df.isna())
```

```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                False             False              False             False   
1                False             False              False             False   
2                False             False              False             False   
3                False             False              False             False   
4                False             False              False             False   
..                 ...               ...                ...               ...   
145              False             False              False             False   
146              False             False              False             False   
147              False             False              False             False   
148              False             False              False             False   
149              False             False              False             False   

     species  
0      False  
1      False  
2      False  
3      False  
4      False  
..       ...  
145    False  
146    False  
147    False  
148    False  
149    False  

[150 rows x 5 columns]
```

Data Analysis

1. mean() : Calculate the mean of a column.
2. median() : Calculate the median of a column.
3. mode() : Calculate the mode of a column.
4. std() : Calculate the standard deviation of a column.
5. var() : Calculate the variance of a column.
6. corr() : Calculate the correlation between two columns.
7. cov() : Calculate the covariance between two columns.

Data Filtering

1. loc[] : Filter rows based on label.
2. iloc[] : Filter rows based on position.
3. query() : Filter rows based on condition.
4. isin() : Filter rows based on values.

Data Sorting

1. sort_values() : Sort a DataFrame by values.
2. sort_index() : Sort a DataFrame by index.

[ ]:
```
# Generate descriptive statistics summary
```

[8]:
```
print(iris_df.describe())
print("Column names %s" % iris_df.columns.values)
print("Size %s" % iris_df.size)
print(iris_df.dtypes)
print(iris_df.shape)
```

```
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)
count        150.000000
mean           1.199333
std            0.762238
min            0.100000
25%            0.300000
50%            1.300000
75%            1.800000
max            2.500000
Column names <ArrowStringArray>
['sepal length (cm)',  'sepal width (cm)', 'petal length (cm)',
  'petal width (cm)',           'species']
Length: 5, dtype: str
Size 750
sepal length (cm)     float64
sepal width (cm)      float64
petal length (cm)     float64
petal width (cm)      float64
species              category
dtype: object
(150, 5)
```

[ ]:
```
# Select specific rows & all columns using loc
```

[9]:
```
print(iris_df.loc[0:8, :])
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2
5                5.4               3.9                1.7               0.4
6                4.6               3.4                1.4               0.3
7                5.0               3.4                1.5               0.2
8                4.4               2.9                1.4               0.2

  species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa
5  setosa
6  setosa
7  setosa
8  setosa
```

[ ]:
```
# Select rows based on a condition
```

[10]:
```
print(iris_df.columns.tolist())
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)', 'species']
```

[11]:
```
print(iris_df.loc[iris_df['sepal length (cm)'] > 7, :])
```

```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
102                7.1               3.0                5.9               2.1
105                7.6               3.0                6.6               2.1
107                7.3               2.9                6.3               1.8
109                7.2               3.6                6.1               2.5
117                7.7               3.8                6.7               2.2
118                7.7               2.6                6.9               2.3
122                7.7               2.8                6.7               2.0
125                7.2               3.2                6.0               1.8
129                7.2               3.0                5.8               1.6
130                7.4               2.8                6.1               1.9
131                7.9               3.8                6.4               2.0
135                7.7               3.0                6.1               2.3

       species
102  virginica
105  virginica
107  virginica
109  virginica
117  virginica
118  virginica
122  virginica
125  virginica
129  virginica
130  virginica
131  virginica
135  virginica
```

[ ]:
```
#    Create new features (e.g., petal area
```

[14]:
```
# Calculate the petal area and add it as a new column
iris_df['petal_area'] = (iris_df['petal length (cm)'] * iris_df['petal width (cm)'])
# Display the first five rows of the modified DataFrame
print(iris_df.head())
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species  petal_area
0  setosa        0.28
1  setosa        0.28
2  setosa        0.26
3  setosa        0.30
4  setosa        0.28
```

[ ]:
```
#  Group by species and calculate mean measurements
```

[15]:
```
print(iris_df.groupby('species').mean())
```

```
            sepal length (cm)  sepal width (cm)  petal length (cm)  \
species
setosa                  5.006             3.428              1.462
versicolor              5.936             2.770              4.260
virginica               6.588             2.974              5.552

            petal width (cm)  petal_area
species
setosa                 0.246      0.3656
versicolor             1.326      5.7204
virginica              2.026     11.2962
```

[ ]:
```
# Filter data for a specific species
```

[16]:
```
setosa_iris = iris_df[iris_df['species'] == 'versicolor']
print(setosa_iris.head())
```

```
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
50                7.0               3.2                4.7               1.4
51                6.4               3.2                4.5               1.5
52                6.9               3.1                4.9               1.5
53                5.5               2.3                4.0               1.3
54                6.5               2.8                4.6               1.5

       species  petal_area
50  versicolor        6.58
51  versicolor        6.75
52  versicolor        7.35
53  versicolor        5.20
54  versicolor        6.90
```

Result:

Program executed successfully. Output obtained and verified.

[ ]:
```
#   Output
```

[2]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)
c_knn = KNeighborsClassifier(n_neighbors=3)
c_knn.fit(x_train, y_train)
y_pred = c_knn.predict(x_test)
print("Accuracy: ", metrics.accuracy_score(y_test, y_pred))
sample = [[2, 2, 2, 2]]
pred = c_knn.predict(sample)
pred_v = [iris.target_names[p] for p in pred]
print(pred_v)
```

```
Accuracy:  0.9777777777777777
[np.str_('setosa')]
```

EXPERIMENT NO : 06

AIM : Program to implement k-NN classification using any standard dataset available in the public domain and find the accuracy of the algorithm.

DESCRIPTION:

1. Import necessary libraries
2. Load the Iris dataset using load_iris
3. x stores the feature values (sepal length, sepal width, petal length, petal width)
4. y stores the target values (species labels)
5. Split the dataset into training (70%) and testing sets (30%) using train_test_split.
6. Create a kNN classifier with 3 nearest neighbours
7. Train the KNN classifier using the training data.
8. Make predictions on the testing data
9. Evaluate the model's performance using accuracy score
10. Define a new sample with feature values
11. Make a prediction on the new sample
12. Get the corresponding species name from the prediction

Result :

Program executed successfully. Output obtained and verified.

[2]:
```python
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

# Load dataset automatically
dataset = sns.load_dataset('iris')

print(dataset.describe())
print(dataset.head())

# Features and Target - converted explicitly to NumPy arrays
x = dataset.iloc[:, :-1].to_numpy()
y = dataset.iloc[:, -1].to_numpy()

# Split dataset
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=0)

# Feature Scaling
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

# Model Training
classifier = GaussianNB()
classifier.fit(x_train, y_train)

# Prediction & Evaluation
y_pred = classifier.predict(x_test)
cm = confusion_matrix(y_test, y_pred)
ac = accuracy_score(y_test, y_pred)

print(cm)
print("Accuracy:", ac)
```

```
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
[[11  0  0]
 [ 0 13  0]
 [ 0  1  5]]
Accuracy: 0.9666666666666667
```

EXPERIMENT NO: 07

AIM: Program to implement Naive Bayes Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm.

DESCRIPTION:

Algorithm:

- **Step 1**: start
- **Step 2**: Importing the standard libraries
- **Step 3**: Load the iris dataset - iris.csv

  The iris dataset contains the following data 50 samples of 3 different species of iris (150 samples total).

  Measurements: Sepal length, sepal width, petal length, petal width

  The format for the data:

  (sepal length, sepal width, petal length, petal width)
- **Step 4**: Define x and y and label the fields
- **Step 5**: Split the dataset into training and testing.
- **Step 6**: Preprocess the dataset using standard scalar. Standard scalar removes the mean and scales each feature / variable to unit variance.
- **Step 7**: Train the data using Gaussian NB model.
- **Step 8**: Test the data using test set.
- **Step 9**: Create the Confusion matrix and find accuracy.
- **Step 10**: Stop.

Result :

Program executed successfully. Output obtained and verified.

[4]:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression
# Importing the dataset
dataset = pd.read_csv('salary_data.csv')
X = dataset.iloc[:, :-1].values #get a copy of dataset exclude last column
y = dataset.iloc[:, 1].values #get array of dataset in column 1st
print(dataset.head())
# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1/3, random_state=0)

#
regressor = LinearRegression()

regressor.fit(X_train, y_train)
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
y_pred

# Visualizing the Training set results
viz_train = plt
viz_train.scatter(X_train, y_train, color='red')
viz_train.plot(X_train, regressor.predict(X_train), color='blue')
viz_train.title('Salary VS Experience (Training set)')
viz_train.xlabel('Year of Experience')
viz_train.ylabel('Salary')
viz_train.show()

# Visualizing the Test set results
viz_test = plt
viz_test.scatter(X_test, y_test, color='red')
viz_test.plot(X_train, regressor.predict(X_train), color='blue')
viz_test.title('Salary VS Experience (Test set)')
viz_test.xlabel('Year of Experience')
viz_test.ylabel('Salary')
viz_test.show()

# Predicting the result of 5 Years Experience
y_pred = regressor.predict([[5]])
print(y_pred)
```

EXPERIMENT NO: 08

AIM: Program to implement linear regression technique using any standard dataset available in the public domain and evaluate its performance.

Predict Canada's per capita income in year 2020. There is an exercise folder here on github at same level as this notebook, download that and you will find canada_per_capita_income.csv file. Using this build a regression model and predict the per capita income for Canadian citizens in year 2020.

Algorithm:

1. Import necessary libraries
2. Load the dataset from a CSV file named Salary_data.csv into a Pandas DataFrame
3. Extract features (X) from the dataset
4. Extract the target variable (y) from the dataset
5. Split the dataset into training (X_train, y_train) and test sets (X_test, y_test) using train_test_split from Scikit-learn
6. Set the test size to $1/3$ of the dataset and use a random state of 0 for reproducibility
7. Create a Linear Regression object (regressor)
8. Train the model using the training data
9. Create a scatter plot of the training data (X_train vs y_train) using Matplotlib
10. Plot the regression line using the trained model's predictions.
11. Add title, labels and display the plot.
12. Use the trained model to predict the salary for 5 years of experience
13. Print the predicted salary.

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0






```
[72284.54698619]
```

Result :

Program executed successfully. Output obtained and verified.

•[11]:

```
import pandas as pd
import numpy as np
"""Step 2: Import Dataset"""
#data_df = pd.read_csv('CCCP - Sheet1.csv')
data_df.head()
"""Step 3: Define X and y"""
X = data_df.drop(['PE'], axis=1).values
y = data_df['PE'].values
print(X)
print(y)
"""Step 4: Split the dataset into training and test sets"""
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=0
)
"""Step 5: Train the model on the training set"""
# Fitting Simple Linear Regression to the Training set
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(x_train, y_train)
"""Step 6: Predict the test results"""
y_pred = regressor.predict(x_test)
print(y_pred)
# regressor.predict([[14.96, 41.76, 1024.07, 73.17]])
"""Step 7: Evaluate the model"""
from sklearn.metrics import r2_score
print(r2_score(y_test, y_pred))
"""Step 8: Plot the results"""
import matplotlib.pyplot as plt
plt.figure(figsize=(15, 10))
plt.scatter(y_test, y_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('ACTUAL VS PREDICTED')
plt.show()
```

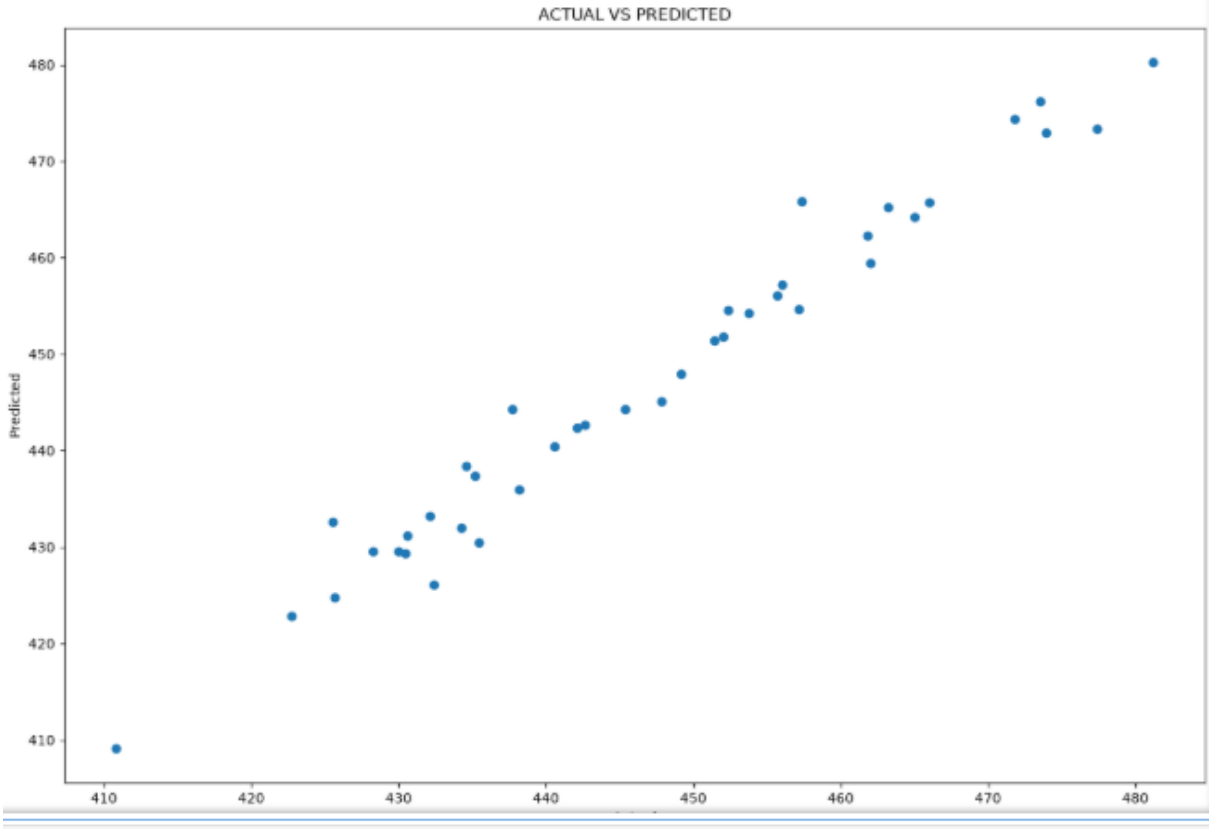
EXPERIMENT NO: 09

AIM: Program to implement multiple linear regression technique using any standard dataset available in the public domain and evaluate its performance.

Algorithm:

1. Import Libraries and Load Dataset
2. Explore Dataset
3. Define features (X) by dropping the target column (PE) from the dataset
4. Define the target variable (y) as the PE column
5. Split the dataset into training (x_train, y_train) and test sets (x_test, y_test) using train_test_split from Scikit-Learn.
6. Set the test size to 33% and use a random state of 0 for reproducibility.
7. Train Simple Linear Regression Model
8. Use the trained model to predict the target variable for the test set.
9. Print the predicted values.
10. Evaluate Model Performance
11. Import r2_score from Scikit-Learn
12. Calculate the R-squared score between actual (y_test) and predicted (y_pred) values.
13. Visualize Results
14. Create a scatter plot of actual (y_test) vs predicted (y_pred) values.
15. Add labels and title.

```
 425.69 412.95 460.4  451.45 458.37 458.11 467.95 441.61 457.26 436.81
 456.8  415.12 430.81 457.37 420.68 432.36 449.79 451.2  440.56 454.27
 465.99 473.95 434.72 428.83 442.09 438.68 450.9  436.88 469.64 461.84
 428.25 447.83 444.21 438.17 433.43 442.28 445.4  461.85 477.41 449.01
 452.87 451.71 428.15 451.7  481.21 463.21 462.04 425.74 442.17 427.07
 429.27 448.31 424.3  471.98 483.06 473.98 432.26 447.16 476.   445.39
 469.47 430.46 457.25 452.06 456.   435.21 452.37 473.54 458.84 419.05
 422.76 437.27 454.37 415.14 433.6  426.47 453.56 457.15 471.69 421.37
 453.76 465.   452.11 442.67 457.82 424.8  434.7  428.37 456.12 437.71]
[440.47718736 457.19278742 437.34968789 474.42238559 476.17985557
 409.09078384 432.00854922 456.11294091 426.15683512 444.26861486
 451.37488427 465.73359768 432.57344477 429.53796282 444.3285939
 480.21928463 424.79231287 465.88752081 464.21599675 459.4176484
 435.97826007 462.30967273 433.2565989  431.14603044 454.22308905
 429.58699654 442.35771065 451.84138283 447.91873761 473.41061755
 472.94566056 442.68062236 454.63497771 445.07007081 438.38994508
 454.53143923 430.46643623 465.20096062 422.86534066 429.34751859]
0.9704010262596563
```



Result

Program executed successfully. Output obtained and verified.

[1]:

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score

# Step 1: Load the dataset
# Using the 20 Newsgroups dataset from scikit-learn
newsgroups = datasets.fetch_20newsgroups(subset='all')

# Step 2: Preprocess the text data
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    newsgroups.data, newsgroups.target,
    test_size=0.3, random_state=42
)

# Step 3: Convert the text data into numerical
# features using TfidfVectorizer
vectorizer = TfidfVectorizer(
    stop_words='english', max_features=5000
)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Step 4: Initialize the SVM classifier
svm = SVC(kernel='linear', random_state=42)

# Step 5: Train the model
svm.fit(X_train_tfidf, y_train)

# Step 6: Predict on the test set
y_pred = svm.predict(X_test_tfidf)

# Step 7: Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=newsgroups.target_names
))
```

EXPERIMENT NO: 10

AIM Program to implement text classification using support vector machine.

DESCRIPTION

1. **Import libraries and load the dataset** - Loads the 20 Newsgroups dataset, a collection of approximately 20,000 newsgroup documents, partitioned across 20 different topics.
2. **Preprocess the text data**
3. **Convert text data into numerical features**
    - **Tfidf Vectorizer:** Transforms text data into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF).
4. **Creates a Support Vector Machine (SVM) classifier** with a linear kernel.
5. **Train the SVM model** on the training data.
6. **Make predictions** on the testing data.
7. **Calculate the accuracy** of the model.
8. **Generate a classification report**, including precision, recall and F1 score.

Accuracy: 0.8760169791298196

Classification Report:

	precision	recall	f1-score	support
alt.atheism	0.87	0.89	0.88	236
comp.graphics	0.70	0.80	0.75	287
comp.os.ms-windows.misc	0.82	0.81	0.81	290
comp.sys.ibm.pc.hardware	0.69	0.76	0.72	285
comp.sys.mac.hardware	0.86	0.82	0.84	312
comp.windows.x	0.87	0.82	0.85	308
misc.forsale	0.80	0.82	0.81	276
rec.autos	0.88	0.90	0.89	304
rec.motorcycles	0.96	0.93	0.94	279
rec.sport.baseball	0.94	0.94	0.94	308
rec.sport.hockey	0.95	0.96	0.96	309
sci.crypt	0.97	0.93	0.95	290
sci.electronics	0.79	0.77	0.78	304
sci.med	0.90	0.92	0.91	300
sci.space	0.93	0.92	0.93	297
soc.religion.christian	0.93	0.97	0.95	292
talk.politics.guns	0.93	0.93	0.93	270
talk.politics.mideast	0.98	0.96	0.97	272
talk.politics.misc	0.90	0.87	0.89	239
talk.religion.misc	0.88	0.78	0.83	196
accuracy			0.88	5654
macro avg	0.88	0.87	0.88	5654
weighted avg	0.88	0.88	0.88	5654

Result

Program executed successfully. Output obtained and verified

[1]:
```
from sklearn.datasets import load_iris
from sklearn import metrics
from sklearn import tree
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)

clf = DecisionTreeClassifier(criterion='entropy')
clf = clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
print("Accuracy :", metrics.accuracy_score(y_test, y_pred))

plt.figure(figsize=(15, 15))
tree.plot_tree(
    clf,
    fontsize=10,
    filled=True,
    rounded=True,
    class_names=iris.target_names,
    feature_names=iris.feature_names,
)
plt.show()
```

EXPERIMENT NO: 11

AIM: Program to implement decision trees using any standard dataset available in the public domain and find the accuracy of the algorithm.

DESCRIPTION:

Decision trees are a popular tool in machine learning, data mining, and statistics that provide a clear and intuitive way to make decisions or predictions based on data. They're flowchart-like structures consisting of nodes representing decisions or tests on attributes, branches representing the outcome of these decisions, and leaf nodes representing final outcomes or predictions.

How Decision Trees Work: Decision trees work by recursively dividing the data according to information gain, which is a measurement of the entropy reduction achieved by splitting on a certain attribute.

Here's a simplified overview of the process:

1. **Select the Best Attribute:** Using a metric like Gini impurity, entropy, or information gain, the best attribute to split the data is selected.
2. **Split the Dataset:** The dataset is split into subsets based on selected attribute.
3. **Repeat the Process:** The process is repeated recursively for each subset, creating a new internal node or leaf node until a stopping criterion is met.

ALGORITHM:

1. Import Libraries
2. Load Iris Dataset
3. Split data into training (70%) and testing sets (30%)
4. Initializes Decision Tree Classifier with entropy criterion
5. Train the classifier on the training data
6. Make predictions on the testing data
7. Calculate model accuracy
8. Plot the Decision Tree
9. Display the plot

```
Accuracy : 0.9555555555555556
```



Result

Program executed successfully. Output obtained and verified

[1]:
```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans

# Load Sklearn IRIS dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Do the scatter plot and see that clusters are evident
plt.scatter(X[:, 1], X[:, 3],
            color='white', marker='o', edgecolor='red', s=50)
plt.grid()
plt.tight_layout()
plt.show()

# Create an instance of K-Means
kmc = KMeans(n_clusters=3, init='random', n_init=10, max_iter=300, tol=1e-04, random_state=0)

# Fit and make predictions
y_kmc = kmc.fit_predict(X)

# Create the K-means cluster plot
plt.scatter(X[y_kmc == 0, 1], X[y_kmc == 0, 3], s=50,
            c='lightgreen', marker='s', edgecolor='black', label='Cluster 1')
plt.scatter(X[y_kmc == 1, 1], X[y_kmc == 1, 3],
            s=50, c='orange', marker='o', edgecolor='black', label='Cluster 2')
plt.scatter(X[y_kmc == 2, 1], X[y_kmc == 2, 3], s=50,
            c='blue', marker='v', edgecolor='black', label='Cluster 3')
plt.scatter(kmc.cluster_centers_[:, 1], kmc.cluster_centers_[:, 3],
            s=250, marker='*', c='red', edgecolor='black', label='Centroids')
plt.legend(scatterpoints=1)
plt.grid()
plt.tight_layout()
plt.show()
```

EXPERIMENT NO: 12

AIM: Program to implement k-means clustering technique using any standard dataset available in the public domain

DESCRIPTION:

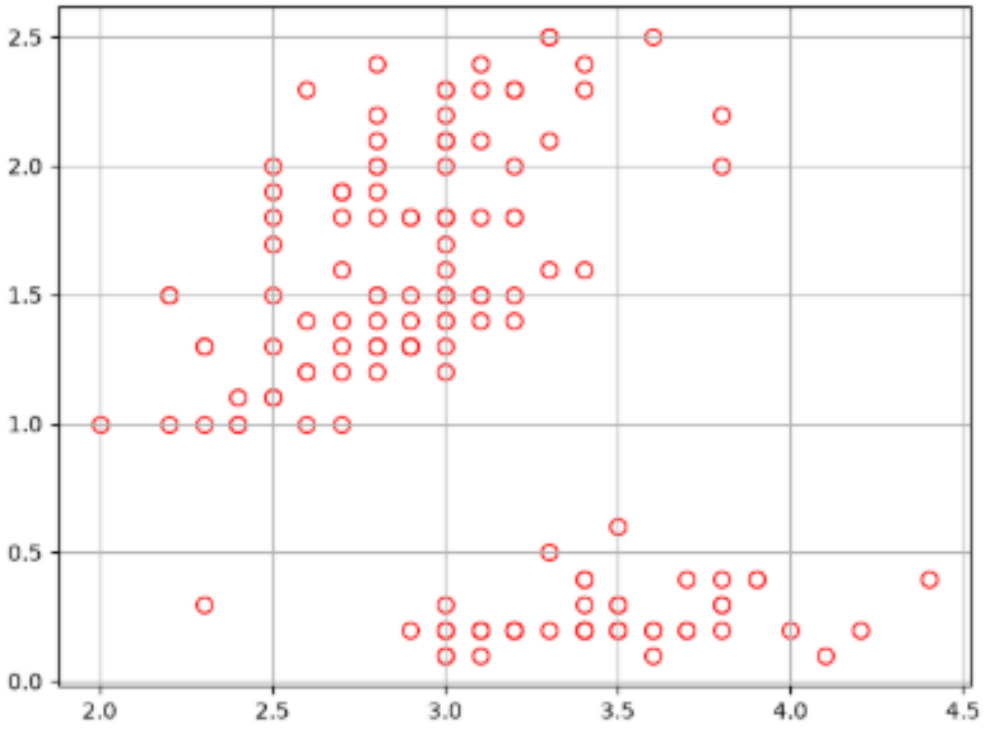
K-Means Clustering is an unsupervised machine learning algorithm used for partitioning data into k distinct clusters based on similarities in features.

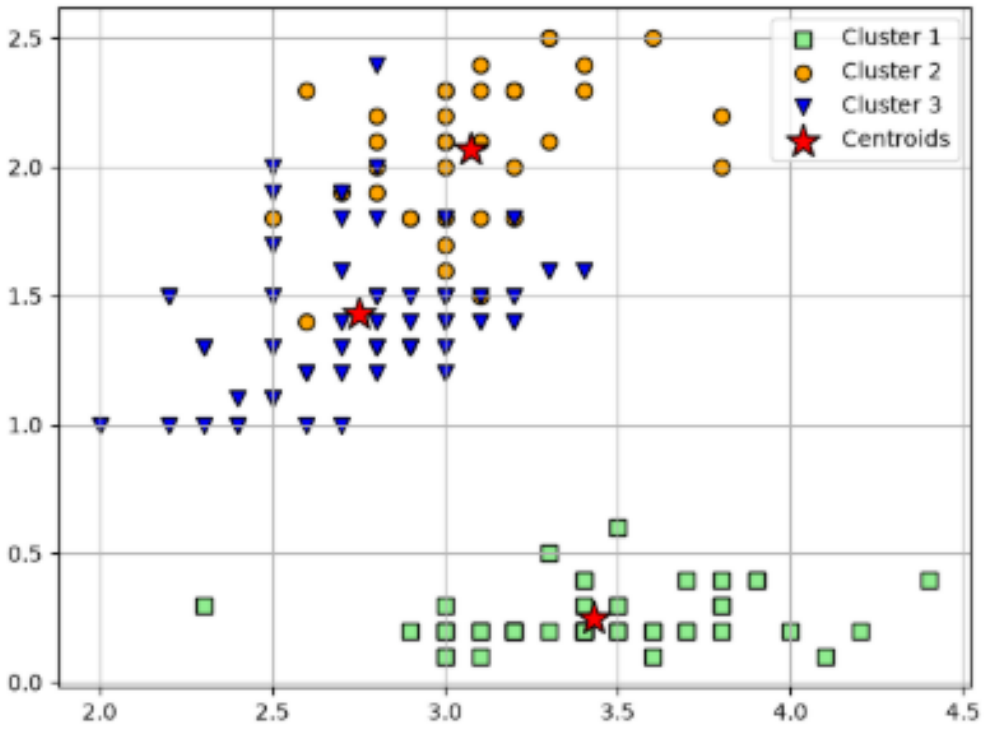
How K-Means Clustering Works:

1. **Initialization:** Randomly select k centroids (center points) for the clusters.
2. **Assignment:** Assign each data point to the closest centroid based on Euclidean distance.
3. **Update:** Update centroids by calculating the mean of all data points assigned to each cluster.
4. **Repeat:** Repeat steps 2-3 until convergence or maximum iterations.

ALGORITHM:

- Import Libraries and load Iris Dataset
- Visualise Iris Dataset
- Create K-Means instance
- Fit K-Means model and predict cluster labels
- Plot clusters with different colors and markers
- Plot centroids
- Display plot


2.5
2.0
1.5
1.0
0.5
0.0
2.0
2.5
3.0
3.5
4.0
4.5


Cluster 1
Cluster 2
Cluster 3
Centroids
2.5
2.0
1.5
1.0
0.5
0.0
2.0
2.5
3.0
3.5
4.0
4.5

Result

Program executed successfully. Output obtained and verified

```python
import nltk

# Download resources (include punkt_tab)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

# Sample text
text = """Machines can learn from past data and automatically improve their performance.
Given a dataset, ML can detect various patterns in the data.
For large organizations, branding is crucial, and targeting a relatable customer base becomes easier.
It is similar to data mining, as both deal with substantial amounts of data."""

# Tokenize the text into words
tokens = nltk.word_tokenize(text)

# Perform Part-of-Speech tagging
pos_tags = nltk.pos_tag(tokens)

# Define chunking grammar
grammar = r"""
    NP: {<DT>?<JJ>*<NN>} # Noun Phrase: Optional Determiner, Adjectives, and Noun
"""

# Create a chunk parser
chunk_parser = nltk.RegexpParser(grammar)

# Parse the tagged words into chunks
chunked = chunk_parser.parse(pos_tags)

print("Chunked Output:")
print(chunked)
```

EXPERIMENT NO: 13

AIM: Implement a program to scrap the webpage of any popular website (Scrapes the titles of articles from a website)

DESCRIPTION:

Web scraping, also known as web data extraction, is the process of automatically collecting and extracting data from websites, web pages, and online documents. This technique involves using specialized software or algorithms to navigate websites, locate and extract specific data, and store it in a structured format.

Types of Web Scraping:

1. **Static Web Scraping:** Extracting data from static websites that do not change frequently.
2. **Dynamic Web Scraping:** Extracting data from dynamic websites that change frequently, such as those using JavaScript or AJAX.
3. **API-Based Web Scraping:** Using APIs (Application Programming Interface) provided by websites to extract data.

Web Scraping Techniques:

1. **HTML Parsing:** Analyzing HTML structure to extract data.
2. **CSS Selectors:** Using CSS selectors to target specific data.
3. **Regular Expressions:** Using patterns to extract data.
4. **JavaScript Rendering:** Executing JavaScript to extract dynamic data.

Web Scraping Tools:

1. Beautiful Soup (Python)
2. Scrapy (Python)
3. Selenium (Python)
4. Octoparse (Visual)

```
Chunked Output:
(S
  Machines/NNS
  can/MD
  learn/VB
  from/IN
  past/JJ
  data/NNS
  and/CC
  automatically/RB
  improve/VB
  their/PRP$
  (NP performance/NN)
  ./.
  Given/VBN
  (NP a/DT dataset/NN)
  ,/,
  ML/NNP
  can/MD
  detect/VB
  various/JJ
  patterns/NNS
  in/IN
  (NP the/DT data/NN)
  ./.
  For/IN
  large/JJ
  organizations/NNS
  ,/,
  (NP branding/NN)
  is/VBZ
  crucial/JJ
  ,/,
  and/CC
  targeting/VBG
  (NP a/DT relatable/JJ customer/NN)
  (NP base/NN)
  becomes/VBZ
  easier/JJR
  ./.
  It/PRP
  is/VBZ
  similar/JJ
  to/TO
  data/VB
  (NP mining/NN)
  ,/,
  as/IN
  (NP both/DT deal/NN)
  with/IN
  substantial/JJ
  amounts/NNS
  of/IN
  data/NNS
  ./.)
```

ALGORITHM:

1. Imports the requests library for making HTTP requests.
2. Specifies the URL of the webpage to scrape.
3. Send a GET request to the specified URL and stores the response.
4. Imports the BeautifulSoup library for parsing HTML.
5. Parses the HTML content of the webpage using the html.parser.
6. Finds all HTML elements with the tag name h2 (assuming article titles are in h2 tags).
7. Loops through each title element.
8. Print the text content of each title element.

RESULT:

Program executed successfully. Output obtained and verified.

[1]:
```python
import requests
from bs4 import BeautifulSoup

# Step 1: Send a request to the webpage
url = 'https://www.geeksforgeeks.org/swift-tutorial/'
response = requests.get(url)

# Step 2: Parse the HTML content
soup = BeautifulSoup(response.content, 'html.parser')

# Step 3: Find all the article titles (assuming they are in <h2> tags)
titles = soup.find_all('h2')

# Step 4: Print the titles
for title in titles:
    print(title.get_text())
```

```
Prerequisites to Learn Swift
Swift Introduction
Swift Data Types
Swift Control Flow
Swift Functions
Swift Collections
Swift OOPs
Swift Additional Topics
Swift 4
What is Swift?
Why Learn Swift?
Applications of Swift
10 Popular Apps Built with Swift
Conclusion
```

EXPERIMENT No: 14

AIM: Demonstrate basic NLP tasks - Tokenization, POS tagging, chunking

DESCRIPTION:

NLP (Natural Language Processing) is a subfield of artificial intelligence (AI) that deals with the interaction between computers and humans in natural language.

Goals of NLP:

1. Enable computers to understand, interpret, and generate human language.
2. Develop algorithms and models to process, analyze, and extract insights from natural language data.

Key NLP Tasks:

1. **Text Preprocessing:** Tokenization, stemming, lemmatization, stopword removal
2. **Part-of-speech (POS) Tagging:** Identifying entities (eg, names, locations) *(Note: this appears to be a slip of the pen, corrected in the definition on following pages)*
3. **Named Entity Recognition (NER):** Identifying entities (eg, names, locations)
4. **Sentiment Analysis:** Determining text sentiment (eg, positive, negative).
5. **Machine Translation:** Translating text from one language to another.

Tokenization

Tokenization is the process of breaking down text into individual words or tokens.

- **Input:** Text
- **Output:** Tokens (words)

Example:

- **Input:** "Machines can learn from past data"
- **Output:** ['Machines', 'can', 'learn', 'from', 'past', 'data']

POS Tagging (Part-of-Speech Tagging)

POS tagging identifies the grammatical category of each word (eg, noun, verb, adjective).

- **Input:** Tokens
- **Output:** Tokens with POS tags

Example:

- **Input:** ['Machines', 'can', 'learn', 'from', 'past', 'data']
- **Output:** [('Machines', NNP), ('can', MD), ('learn', VB), ('from', IN), ('past', NN), ('data', NN)]

Common POS tags:

- NN : Noun
- VB : Verb
- JJ : Adjective
- RB : Adverb
- IN : Preposition
- MD : Modal verb

3 Chunking (Shallow Parsing)

Chunking groups tokens into phrases based on grammatical relationships.

- **Input:** Tokens with POS tags
- **Output:** Chunked phrases

Example:

- **Input:** [('Machines', NNP), ('can', MD), ('learn', VB), ('from', IN), ('past', NN), ('data', NN)]
- **Output:** [(NP 'Machines'), (VP 'can learn'), (PP 'from past data')]

Chunking identifies phrases such as:

- - NP : Noun phrase
- - VP : Verb phrase
- - PP : Prepositional phrase
- - ADJP : Adjective phrase
- - ADVP : Adverb phrase

ALGORITHM:

1. Download the necessary NLTK resources:
    - - punkt : Tokenization models for sentence splitting and word tokenization.
    - - averaged_perceptron_tagger : Part-of-speech (POS) tagging model.
2. Define a sample text paragraph.
3. Split the text into individual words or tokens using word_tokenize.
4. Assign POS tags to each token using pos_tag.
5. Define a chunking grammar using regular expressions.
6. Create a chunk parser using the defined grammar.
7. Parse the POS-tagged tokens into chunks based on the grammar.
8. Display the chunked structure of the text.

RESULT:

Program executed successfully. Output obtained and verified.